Formula Extraction: Premise, Field Map, and Options

A two-page conversation brief for Taru

June 13, 2026

1 Premise

We care about formula extraction because scientific and educational documents are still hard to convert into useful machine-readable math at scale. The interesting opportunity is not "OCR but with another VLM." It is **efficient, verifiable structured extraction from scientific documents**, beginning with formulas. Docling is a useful integration target, but not the definition of the problem.

The seed idea has three plausible levels: a review/benchmark, a hard dataset/evaluation suite, or a small efficient formula adapter/model that converts formula crops/pages into LaTeX, preserves provenance, exposes confidence and validation metadata, and later supports a formal subset that can be checked in Lean.

Important framing: Lean does not validate arbitrary LaTeX. LaTeXis display syntax; Lean is typed formal math. The practical stack is:

$$\text{image} \rightarrow \text{LaTeX} \rightarrow \text{canonical LaTeX} \rightarrow \text{render/visual checks} \rightarrow \text{optional typed Lean check}$$

2 What The Field Has Done Well Since 2023

1. Scientific PDF-to-markup became real. Nougat showed that academic PDFs can be converted into markup with formulas, attacking the "PDF loses semantics" problem directly. It proved demand and set the scientific-document framing, but it is academic-document-specialized and not a full general document system.

2. General document parsers got much stronger. Docling, MinerU, Dolphin, PaddleOCR-VL, and olmOCR pushed the field toward full document conversion: reading order, layout, tables, formulas, charts, lists, scanned pages, and Markdown/HTML-like outputs. The best systems now combine learned models with careful preprocessing/postprocessing and benchmark discipline.

3. Small/compact VLMs became credible. SmolDocling is the key Docling-native example: a 256M VLM generating DocTags with layout and content. GOT-OCR2.0 is another strong signal: a 580M unified OCR-2.0 model for text, formulas, tables, charts, geometry, and formatted outputs. PaddleOCR-VL targets a compact 0.9B multilingual parser. The lesson is that good visual tokenization and data curation can beat brute force for document tasks.

4. Synthetic data works when it is aligned and checkable. SynthFormulaNet and SynthCodeNet are large, controlled image-to-text corpora. ChartNet goes further by aligning chart image, plotting code, source table, summary, and QA. olmOCR 2 is especially relevant because it uses unit-test-like verifiable rewards for document OCR, with gains in formulas, tables, and multi-column layouts.

5. The frontier is moving from OCR to structured visual code. GOT, DeepSeek-OCR, MOCR/dots.mocr, and chart/table systems all point the same way: document elements are not just pixels to transcribe; they are visual programs to recover. Formulas are one of the purest cases of this problem.

3 What Is Already Done Super Well

Area	Strong existing work
Document pipeline	Docling gives a clean, extensible document conversion stack. We should build with it, not around it.
Formula/code baseline	CodeFormulaV2 already handles image crops of formulas/code and is trained on SynthFormulaNet/SynthCodeNet.
Full-page conversion	SmolDocling, olmOCR, MinerU, Dolphin, and PaddleOCR-VL show strong page-level extraction and reading-order work.
Efficient VLM design	Vary-toy, SmolDocling, GOT, DeepSeek-OCR, and PaddleOCR-VL show compact models can work if visual tokens/data are good.
Synthetic supervision	SynthFormulaNet, SynthCodeNet, ChartNet, and olmOCR-style synthetic unit tests show scalable supervision is possible.

4 What Still Does Not Feel Solved

Formula quality is not just exact-match OCR. Two LaTeXstrings may render identically; one string may compile but mean the wrong thing; another may be visually right but semantically under-specified. We need layered evaluation: canonical string, compile/render, visual similarity, and optional semantic checks.

Real scientific PDFs are messy. Synthetic formulas are clean. Real papers have equation numbers, line breaks, multi-line alignments, low-resolution scans, font issues, cropped symbols, ambiguous glyphs, theorem context, and notation defined in prose.

Model outputs rarely carry trust metadata. A downstream agent needs to know: did this formula compile, did it visually match the crop, was it low confidence, did alternate candidates exist, and is this formula eligible for Lean validation?

Formal validation is underused and often misunderstood. Lean can be powerful, but only after a typed semantic parse. The right move is a narrow high-value Lean subset, not pretending every formula crop can be auto-formalized.

There is no obvious practical formula extraction benchmark. Existing systems report broad document metrics. A hard, curated, formula-centered benchmark with validation traces would be useful even before model training.

5 Proposed Contribution

The gap is not "make another OCR model." A potentially useful contribution is a **review/benchmark**: compare VLM and non-VLM formula extraction approaches for local scientific/educational document workflows, and identify where CodeFormulaV2 or current SOTA is best versus where another approach may be cheaper, faster, or more reliable.

> **Target artifact:** a formula extraction benchmark and adapter study with validation traces: image-to-LaTeX, canonicalization, compile/render checks, visual equivalence, time-to-parse, memory footprint, layout failure taxonomy, and a small Lean-checkable subset.

6 Targets We Should Optimize

Target	Metric audiences may care about
On-device viability	Model size, peak RAM, CPU/MPS latency, cold start, no cloud dependency.
Faster parsing	Total PDF parse time, seconds/page, formulas/second, overhead versus a base PDF/OCR pipeline.
Higher accuracy	Normalized LaTeXexact match, symbol error rate, structure/tree similarity, render similarity, compile rate.
Layout handling	Formula detection recall, bbox quality, inline/display split, multi-line equations, equation numbers, reading order.
Trust metadata	Confidence calibration, abstention quality, alternate candidates, compile/render pass, visual-match score, Lean-valid subset.

The practical bar is a Pareto curve, not one score: **accuracy vs latency vs memory vs trust**. Taru's point is central: an accurate system that takes 30 minutes for 10 pages is not useful for real bulk document workflows.

7 Possible First Milestone

1. Sample and inspect SynthFormulaNet plus a small set of real math-heavy PDFs.
2. Build a validator: parse target, render LaTeX, compare to source crop, tag failures.
3. Run CodeFormulaV2, SmolDocling/Granite-Docling, one larger VLM baseline, and one non-VLM/specialized formula OCR baseline if available.
4. Create a hard 500–2,000 example benchmark stratified by formula type.
5. Report accuracy, parse time, memory, and layout failures on 10-page, 50-page, and math-heavy stress PDFs.
6. Optionally wrap best baseline/cascade as a document-parser enrichment adapter with validation metadata.
7. Add a tiny Lean-valid pilot: elementary algebra/calculus/theorem statements with explicit assumptions.

8 Questions For Taru

Decision questions: benchmark vs adapter vs validation tool? arXiv/math PDFs vs textbooks vs scanned papers? local speed vs top accuracy vs trust metadata? which Lean slice is realistic first? Paper shortlist for follow-up: CodeFormulaV2, SynthFormulaNet, SmolDocling, Nougat, GOT-OCR2.0, DeepSeek-OCR/2, olmOCR 2, DocTron-Formula, ChartNet.